

Projet Pokémon Web Game

Documentation du schéma de base de données — v2

Stack : React + TypeScript / Node.js / MySQL 8 (InnoDB)

14 tables. Structure et nommage entièrement en anglais.

Périmètre : joueurs, profil, pokémon, équipe, items, fragments, évolution, équilibrage

Hors périmètre : gacha, monnaies, PvP, compétences, badges, mode histoire, table des types

Quatre concepts à retenir avant de lire.

1. Template vs Instance. pokemon est un catalogue d'espèces (statique, ~251 lignes). pokemon_instances contient les pokémon possédés. Trois Dracaufeu = 3 instances, 1 seule ligne catalogue. Même logique pour les items.

2. Deux axes de progression séparés. Les pierres font évoluer l'espèce (Salamèche → Reptincel). Les fragments font monter les étoiles de puissance. Ils ne se mélangent jamais.

3. Les fragments appartiennent à la ligne évolutive, pas à l'espèce. Salamèche, Reptincel et Dracaufeu puisent dans le même pot.

4. Aucune stat calculée n'est stockée. Les stats finales sont recalculées à la volée par le service. Seuls les coefficients vivent en base.

Partie 1 — Le joueur et son profil

1. avatars

Catalogue des avatars débloqués. Données statiques.

Champ	Type	Null	Rôle
id	INT AI	non	Clé primaire.
name	VARCHAR(50)	non	Nom affiché.
image_url	VARCHAR(255)	non	Chemin de l'image.

2. users

Les comptes joueurs. Racine de toutes les possessions.

Champ	Type	Null	Rôle
id	INT AI	non	Clé primaire.
email	VARCHAR(255)	non	Identifiant de connexion. UNIQUE .
password_hash	VARCHAR(255)	non	Hash bcrypt. Jamais le mot de passe en clair . bcrypt produit 60 caractères, on garde de la marge si l'algo change.
display_name	VARCHAR(50)	non	Le pseudo affiché. Le "NAME" du wireframe PROFIL.
country	CHAR(2)	oui	Le "FLAG" du wireframe. Code ISO 3166-1 alpha-2 : FR, JP, US. Pas de table pays : ces codes sont universels et immuables. Le front affiche le drapeau correspondant.
level	SMALLINT	non	Le "LVL" du wireframe. Niveau du <i>joueur</i> , distinct du niveau de ses pokémon.
xp	INT	non	Expérience du joueur.
active_avatar_id	INT (FK)	oui	L'avatar actuellement affiché. NULL = avatar par défaut.
created_at	DATETIME	non	Date d'inscription.

3. user_avatars

Quels avatars un joueur a débloqués. **Aucune limite en base.**

Champ	Type	Null	Rôle
id	INT AI	non	Clé primaire.
user_id	INT (FK)	non	Le propriétaire.
avatar_id	INT (FK)	non	L'avatar débloqué.
unlocked_at	DATETIME	non	Date de déblocage.

Contrainte : UNIQUE (user_id, avatar_id) → pas de doublon

Pourquoi pas de limite à 4 en base. Ton wireframe montre 4 vignettes. Mais contraindre à 4 en base t'obligerait à gérer "l'inventaire est plein, tu perds le 5e" — une règle pénible pour rien. **La limite de 4 est un choix d'affichage du front.** Le joueur débloque autant d'avatars qu'il veut, il en affiche 4, il en active 1.

Règle R8 : users.active_avatar_id doit exister dans user_avatars pour ce joueur. C'est une contrainte inter-tables, **non exprimable en SQL** — à vérifier dans le service avant chaque UPDATE.

Badges : volontairement absents. Le système n'est pas défini (d'où viennent-ils ? succès, ligue PvP, achat ?). Modéliser un système non pensé garantit de le refaire. Le jour venu : badges (catalogue) + user_badges (qui

a quoi, quand). Deux tables triviales.

Partie 2 — Les pokémon

4. stones

Catalogue des pierres d'évolution. **Une pierre par type Pokémon** (Feu, Eau, Plante, Électrik...), soit 17 lignes pour les générations 1 et 2. Décision de design : on crée une pierre pour *chaque* type, même ceux qui n'en avaient pas dans le jeu original.

Champ	Type	Null	Rôle
id	INT AI	non	Clé primaire.
name	VARCHAR(50)	non	Ex : "Fire Stone".
pokemon_type	VARCHAR(20)	non	Le type correspondant. UNIQUE : une pierre par type.

5. pokemon

Le catalogue des espèces. Rempli une fois, jamais modifié par un joueur. Contient les stats de base et toute la logique d'évolution.

Champ	Type	Null	Rôle
id	INT AI	non	Clé primaire.
name	VARCHAR(50)	non	Ex : "Charmander".
evolution_line_id	INT	non	Champ clé. Regroupe une famille : Salamèche, Reptincel et Dracaufeu partagent la même valeur. C'est ce qui rend le pot de fragments commun à toute la ligne. Volontairement une simple colonne plutôt qu'une table dédiée.
evolution_stage	TINYINT	non	0, 1 ou 2. Double usage : afficher le stade, et calculer la valeur d'un doublon sacrifié (règle R1).
evolves_into_id	INT (FK)	oui	Pointe vers l'espèce suivante. Auto-référence sur pokemon . id. NULL pour un stade final.
stone_id	INT (FK)	oui	Quelle pierre est requise. NULL si pas d'évolution.
stone_cost	SMALLINT	oui	Combien de pierres. NULL si pas d'évolution.
type_primary	VARCHAR(20)	non	Type principal.
type_secondary	VARCHAR(20)	oui	Type secondaire. NULL si mono-type. Fidèle à la Gen 1-2.
base_atk / base_hp base_def / base_speed	SMALLINT	non	Stats de base de l'espèce. Un Dracaufeu et un Roucool n'ont pas les mêmes. Étoiles, niveau et items s'ajoutent par-dessus (règle R7).
generation	TINYINT	non	1 ou 2 pour l'instant.

Deux pièges sur cette table.

1. L'auto-référence. evolves_into_id pointe vers pokemon . id — la table se référence elle-même. Lors du remplissage du catalogue, insère **Dracaufeu avant Reptincel**, sinon la clé étrangère échoue. Ou fais les INSERT sans ce champ, puis un UPDATE.

2. evolution_line_id n'est contraint par rien. C'est le prix de la simplicité (pas de table dédiée). Si tu tapes 44 au lieu de 4 pour Dracaufeu, MySQL accepte — et ton Dracaufeu se retrouve seul dans un pot de fragments. **Vérifie tes lignes évolutives à l'insertion**, c'est le seul endroit où ça peut déraiper.

6. pokemon_instances

Les pokémon réellement possédés. C'est ici que vit tout ce qui appartient au joueur.

Champ	Type	Null	Rôle
id	INT AI	non	Clé primaire.
user_id	INT (FK)	non	Le propriétaire. ON DELETE CASCADE.
pokemon_id	INT (FK)	non	L'espèce. Ce champ change lors d'une évolution : même ligne, même instance, mais elle pointe vers une autre espèce.
stars	TINYINT	non	Étoile actuelle, 1 à 5. CHECK en base.
star_progress	SMALLINT	non	Fragments investis dans l'étoile en cours . Séquentiel : au seuil, stars += 1 et star_progress repart à 0. Le front dessine 5 branches, la base ne connaît qu'un entier.
is_shiny	BOOLEAN	non	Cosmétique pur, aucun bonus. Coût fixe (game_settings).
level	SMALLINT	non	Niveau du pokémon. Défaut 1.
xp	INT	non	Expérience. Le seuil du niveau suivant se lit dans level_costs.
obtained_at	DATETIME	non	Date d'obtention.

Note : le champ nickname a été supprimé en v2. **Aucune stat calculée n'est stockée ici** — voir la règle R7.

Partie 3 — Équipe et équipement

7. team_slots

L'équipe unique, 6 slots maximum. **Pas de table** teams : un joueur n'a qu'une équipe, les slots pointent directement sur user_id. Une équipe incomplète = moins de 6 lignes. Jamais de ligne vide.

Champ	Type	Null	Rôle
id	INT AI	non	Clé primaire.
user_id	INT (FK)	non	Le propriétaire de l'équipe.
slot_position	TINYINT	non	1 à 6. Détermine l'ordre d'attaque , gauche à droite. CHECK en base.
pokemon_instance_id	INT (FK)	non	Pointe vers une instance , jamais vers le catalogue. C'est <i>ton</i> Dracaufeu ■4 équipé, pas l'espèce.

UNIQUE (user_id, slot_position) → pas deux pokémon sur le même slot
UNIQUE (user_id, pokemon_instance_id) → pas le même pokémon deux fois

8. item_templates

Le catalogue des items. Un item se définit par **deux axes indépendants** : la **rareté** détermine la *puissance*, le **mode** détermine le *comportement*.

Champ	Type	Null	Rôle
id	INT AI	non	Clé primaire.
name	VARCHAR(100)	non	Nom affiché.
category	ENUM(att, def, speed, spe)	non	Le slot typé où l'item se monte. Les 4 boutons ATT/SPE/DEF/SPEED du wireframe.
mode	ENUM (9 val.)	oui	NULL pour att et speed (un seul comportement). def : hp ou taunt. spe : crit, anticrit, ou 5 modes de heal (heal_left, heal_right, heal_random, heal_lowest, heal_adjacent).
rarity	ENUM(common, rare, ultra_rare, legendary, mythic)	non	Détermine la puissance du boost.
boost_value	SMALLINT	non	La valeur du boost, fonction de la rareté.

Pourquoi deux axes et pas un seul. Deux items Spécialiste légendaires peuvent être radicalement différents : l'un fait crit, l'autre heal à gauche. La rareté seule ne définit donc pas un item. Le mode porte le comportement, la rareté porte la puissance.

C'est l'équipement qui crée les rôles, pas la position dans l'équipe. Un même pokémon devient tank, DPS ou healer selon ce qu'on lui monte. Le méta-jeu est là.

9. item_instances

Les items possédés. **Réserve et équipé vivent dans la même table.** Pas de table user_items séparée.

Champ	Type	Null	Rôle
id	INT AI	non	Clé primaire.
user_id	INT (FK)	non	Le propriétaire.
item_template_id	INT (FK)	non	Le modèle dans le catalogue.

Champ	Type	Null	Rôle
category	ENUM(att, def, speed, spe)	non	Dénormalisation volontaire : copie de item_templates.category. Voir l'encadré.
pokemon_instance_id	INT (FK)	oui	Le champ central. NULL = en réserve. Rempli = équipé sur ce pokémon. ON DELETE SET NULL : si un pokémon disparaît, ses items retournent en réserve au lieu d'être détruits.
item_level	TINYINT	non	Défaut 1. Champ dormant : l'évolution d'items n'est pas implémentée, mais le champ évite une migration le jour où on l'ajoute.
obtained_at	DATETIME	non	Date d'obtention.

UNIQUE (pokemon_instance_id, category)

À comprendre absolument : NULL n'est jamais égal à NULL en MySQL.

La contrainte UNIQUE (pokemon_instance_id, category) empêche un pokémon d'avoir deux items de la même catégorie — c'est ce qui matérialise les slots typés.

Mais pour les items en réserve, pokemon_instance_id vaut NULL, et comme deux NULL ne sont pas égaux, **la contrainte ne s'applique pas**. Un joueur peut avoir 500 items attaque en réserve sans conflit. C'est exactement le comportement voulu, et c'est un piège classique — dans un sens comme dans l'autre.

Pourquoi dénormaliser category. La contrainte doit porter sur la catégorie, mais celle-ci vit sur item_templates, et MySQL ne sait pas contraindre sur une colonne d'une autre table. On copie donc la valeur. Le risque habituel — la copie qui diverge — n'existe pas ici : **la catégorie d'un item ne change jamais**.

L'alternative serait un check dans le service Node, mais ce serait une règle en JS au lieu d'une garantie en base : un oubli à un seul endroit et tes données sont corrompues.

Partie 4 — Ressources de progression

10. user_fragments

Le pot de fragments. **La clé est evolution_line_id, pas pokemon_id.** C'est le point le plus important de cette table.

Champ	Type	Null	Rôle
id	INT AI	non	Clé primaire.
user_id	INT (FK)	non	Le propriétaire.
evolution_line_id	INT	non	La famille. Un pot pour Salamèche + Reptincel + Dracaufeu.
quantity	INT	non	Fragments disponibles.

UNIQUE (user_id, evolution_line_id) → un pot par joueur et par ligne

Pourquoi par ligne évolutive et pas par espèce. Si les fragments étaient attachés à l'espèce, voici ce qui arriverait : tu fais évoluer ton Salamèche ■3 en Reptincel. Pour monter ses étoiles il faudrait maintenant des fragments *de Reptincel*... que tu n'obtiendrais qu'en tirant des Reptincel au gacha. Si le gacha ne donne que des Salamèche, **ton Reptincel ne pourrait plus jamais progresser.** Piège classique du genre. Le pot par ligne le supprime.

11. user_stones

Les pierres possédées. Même logique de pot.

Champ	Type	Null	Rôle
id	INT AI	non	Clé primaire.
user_id	INT (FK)	non	Le propriétaire.
stone_id	INT (FK)	non	Le type de pierre.
quantity	INT	non	Le "12/16" du wireframe = quantity / pokemon.stone_cost.

Partie 5 — Tables d'équilibrage

Ces trois tables n'ont **aucune relation** avec le reste. Ce sont des tables de référence. Leur raison d'être : **tu ajustes ton jeu avec des UPDATE SQL, jamais en modifiant du JavaScript**, et aucun nombre magique ne traîne dans ton code.

12. star_costs

Étoile	Par branche	Total (base)	Cumul depuis ■1
■1	5	25	25
■2	10	50	75
■3	20	100	175
■4	50	250	425
■5	100	500	925

La colonne "par branche" est indicative — elle n'est **pas** stockée. Les 5 branches sont un dessin du front, la base ne connaît qu'un total. **925 fragments** pour un ■5 : soit ~925 doublons de Salamèche, soit ~308 doublons de Dracaufeu (voir R1).

13. level_costs

Barème XP des pokémon. `xp_required` = XP cumulée pour *atteindre* ce niveau. **La courbe de coût est exponentielle, le gain de stat est linéaire** (voir R7). Rempli jusqu'au niveau 10 dans le SQL, à compléter jusqu'au niveau max souhaité.

14. game_settings

Constantes d'équilibrage, en clé/valeur. DECIMAL(10,4) pour accepter les coefficients.

Clé	Valeur	Rôle
shiny_fragment_cost	1000	Coût du shiny. Fixe pour toutes les espèces : la rareté du drop suffit à rendre la tâche difficile.
star_stat_coeff	0.10	Gain de stat par étoile.
level_stat_coeff	0.02	Gain de stat par niveau.
team_size	6	Slots dans une équipe.
equip_slots	4	Slots d'équipement par pokémon.
max_stars	5	Étoile maximum.

Règles de gestion

Ces règles **ne sont pas exprimables en SQL**. Elles vivent dans les services Node.js, et chacune doit s'exécuter dans une **transaction** : si une étape échoue, tout est annulé. Sans transaction, un joueur peut perdre ses fragments sans gagner son étoile — le genre de bug qui fait fuir les joueurs.

R1 — Valeur d'un doublon

Sacrifier un doublon rapporte (**evolution_stage + 1**) fragments dans le pot de sa ligne.

Salamèche (stage 0) → 1 fragment
Reptincel (stage 1) → 2 fragments
Dracaufeu (stage 2) → 3 fragments

Exemple : 1 Salamèche + 2 Dracaufeu = 1 + (3 × 2) = **7 fragments**

Un doublon n'a pas une valeur fixe : son stade détermine ce qu'il rapporte. Un tirage de Dracaufeu vaut trois fois un Salamèche, même sacrifié.

R2 — Montée d'étoile

Séquentielle et obligatoire : **impossible de viser ■3 avant d'avoir terminé ■2**. Coût lu dans star_costs. On incrémente star_progress ; au seuil, stars += 1 et star_progress = 0.

R3 — Évolution d'espèce

Consomme pokemon.stone_cost pierres du type pokemon.stone_id. L'instance change de pokemon_id vers evolves_into_id.

Conservés : étoiles, progression, niveau, xp, shiny, items équipés.

Non consommés : les fragments. Ils servent uniquement aux étoiles.

R4 — Shiny

Coût fixe : game_settings.shiny_fragment_cost (1000). Identique pour toutes les espèces. Bascule is_shiny à TRUE. **Purement cosmétique, aucun bonus de stat.**

R5 — Équipement

Équiper : UPDATE item_instances SET pokemon_instance_id = ? WHERE id = ?

Déséquiper : UPDATE item_instances SET pokemon_instance_id = NULL WHERE id = ?

Aucun déplacement entre tables, donc **aucun risque de dupliquer ou perdre un item**. La contrainte UNIQUE garantit le reste.

R6 — Combat

Format : 6 contre 6, alignés de gauche à droite.

Initiative entre joueurs : somme des vitesses d'équipe.

Ordre d'action interne : slot_position 1 → 6. La vitesse individuelle ne joue aucun rôle ici.

Ciblage : s'il existe des pokémon avec Taunt, cible aléatoire parmi eux. Sinon, cible aléatoire parmi tous les ennemis vivants.

Heal : le mode de l'item détermine la cible. Montant = attaque du pokémon convertie en PV.

Les rôles viennent des items, pas de la position.

Le calcul de combat se fait exclusivement côté serveur. Jamais côté client. Un client peut être modifié par le joueur : toute logique de jeu qui y vivrait serait immédiatement exploitable. Le front affiche le résultat, il ne le calcule pas.

R7 — Calcul des stats

```
stat = base
      × (1 + star_stat_coeff × (stars - 1))
      × (1 + level_stat_coeff × (level - 1))
      + SOMME(boosts des items équipés)
```

Appliqué à atk, hp, def et speed. Exemple avec Dracaufeu (base_atk 84) :

État	Calcul	ATK finale
■1, niveau 1	84	84
■3, niveau 20	$84 \times 1.20 \times 1.38$	139
■5, niveau 50	$84 \times 1.40 \times 1.98$	233
■5, niv. 50 + 4 items légendaires	$233 + (4 \times \sim 25)$	~333

Pourquoi les stats ne sont pas stockées. Dès qu'on stocke une stat calculée, il faut la recalculer à chaque équipement, chaque montée d'étoile, chaque niveau, chaque évolution. Le jour où on en oublie un, on a des stats fausses en base, **silencieusement**. On calcule donc à la volée dans le service. Seuls les **coefficients** vivent en base — eux, c'est de l'équilibre.

Pourquoi le gain n'est pas exponentiel. Un gain exponentiel ($\text{base} \times 1.15^{\text{level}}$) donnerait 108 000 ATK au niveau 50 et dépasserait le milliard au niveau 100 : combats illisibles, SMALLINT explosé. Ce que font réellement les gacha : **la courbe de coût est exponentielle, le gain de stat est linéaire**. C'est ça qui crée la progression — tu montes régulièrement, mais chaque palier demande plus d'effort. La progression x4 entre ■1 niv.1 et ■5 niv.50 équipé est saine.

R8 — Avatars

`users.active_avatar_id` doit exister dans `user_avatars` pour ce joueur. Contrainte inter-tables, non exprimable en SQL : à vérifier dans le service avant chaque UPDATE.

La requête centrale

Récupérer l'équipe complète d'un joueur avec stats de base et items équipés. C'est la première requête que tu écriras, et la plus utilisée. **4 jointures.**

```
FROM team_slots ts -- filtré sur user_id
  JOIN pokemon_instances pi ON ... -- 1 : l'instance
  JOIN pokemon p ON ... -- 2 : stats de base
  LEFT JOIN item_instances ii ON ... -- 3 : items équipés
  LEFT JOIN item_templates it ON ... -- 4 : boosts et modes
ORDER BY ts.slot_position
```

Deux choses à ne pas rater.

1. Les LEFT JOIN sur les items sont obligatoires. Un pokémon sans item existe. Avec un INNER JOIN il disparaîtrait du résultat : ton équipe de 6 en afficherait 4. Bug classique et pénible à diagnostiquer.

2. La requête renvoie jusqu'à 24 lignes, pas 6. Chaque pokémon est dupliqué autant de fois qu'il a d'items (6 × 4). C'est le fonctionnement normal d'une jointure. Ton code Node devra regrouper ces lignes en 6 objets — soit en JS, soit avec deux requêtes séparées, soit avec JSON_ARRAYAGG en MySQL 8. Ne sois pas surpris.

Note d'évolution : aujourd'hui team_slots porte user_id directement, donc pas de jointure vers users. Le jour où tu ajoutes des équipes multiples (une PvE, une PvP), la table teams réapparaît et s'intercale : une jointure de plus. C'est le seul endroit qui bougerait — le schéma encaisse le changement sans rien casser d'autre.

Ce qui n'est pas encore modélisé

Système	Ce qu'il faudra ajouter
Gacha / packs	packs, user_packs. Taux de rareté, pity system. L'ouverture crée des pokemon_instances ; les doublons alimentent user_fragments via R1.
Monnaies	Pokédollars et Masterballs. Deux colonnes sur users suffisent — pas besoin d'une table.
Badges	badges + user_badges. Système non défini, donc non modélisé. Le front peut itérer sans la base.
Compétences	L'onglet COMPÉTENCE du wireframe. Règles non définies. Probablement skills + instance_skills, débloquées par les étoiles.
PvP	pvp_matches, Elo, ligues, récompenses. Le combat étant simulé serveur, on stocke le résultat et le déroulé.
Table des types	type_effectiveness, ~289 lignes (17 × 17). Reporté : l'item anticrit donne pour l'instant une résistance simple contre un type, sans matrice.
Mode histoire	Chapitres, combats PNJ, récompenses.
Évolution d'items	item_level est déjà en place, à 1. Mécanique à définir.

Ordre de construction recommandé

1. Auth (register / login, JWT, bcrypt) — le socle, réutilisé partout.
2. Remplir les catalogues : stones, pokemon, item_templates, avatars, et les 3 tables d'équilibrage.
3. Une verticale complète et unique : afficher l'inventaire d'un joueur, du front à la base.
4. L'écran TEAM : composer les 6 slots.
5. Équiper / déséquiper un item.
6. Évolution et étoiles.
7. Le reste (gacha, PvP, histoire).

Ne construis pas dans l'ordre du document de game design. Une verticale complète qui fonctionne de bout en bout t'apprend plus, et te motive davantage, que six tables à moitié câblées. Le PvP, le mode histoire et la monétisation sont des projets à part entière — les garder pour plus tard n'est pas de la lâcheté, c'est la seule façon de finir.

Prérequis technique : les CHECK constraints exigent MySQL 8.0.16+. En dessous, elles sont ignorées **silencieusement** — vérifie ta version avant d'exécuter le schéma.